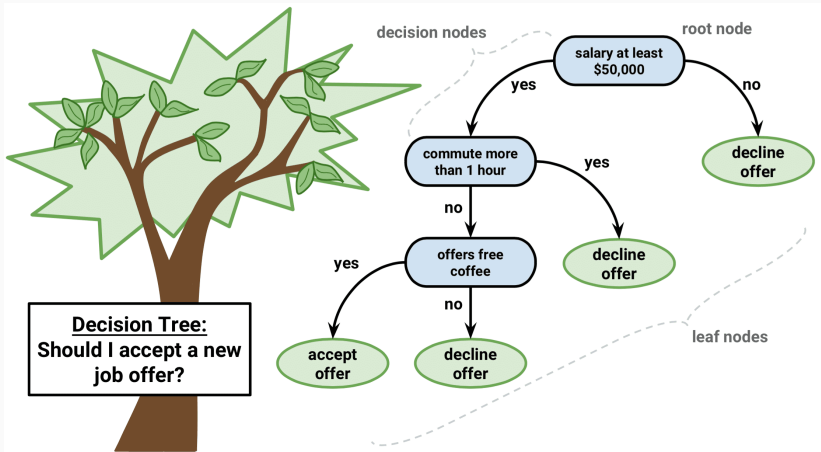


Supervised Learning: Classification

Tree-based methods and Support Vector Machines

Mahdi Shafiee Kamalabad and Maarten Cruyff

Classification trees



Today's topics

1. **Classification Trees**

Predict a class by splitting the data into regions using feature-based rules.

2. **Pruning**

Simplifies trees by removing unnecessary splits to reduce overfitting.

3. **Bagging and Random Forests**

Combine many trees to improve accuracy and reduce variance.

4. **Boosting**

Builds trees sequentially to improve prediction by focusing on difficult cases.

5. **Support Vector Machines (SVMs)**

Find a boundary that separates classes with the largest margin.

Classification Trees

Classification trees predict a class by repeatedly splitting the data into smaller groups.

1. Use **recursive binary splitting**
2. Choose splits that make the resulting nodes as **pure** as possible
3. Assign a class label based on the **majority class** in each terminal node

Node purity:

A node is more pure when most observations in it belong to the same class.

Common purity measures:

- **Gini index**
- **Deviance**

Example: Carseats (binary classification)

- **Dataset:** ISLR::Carseats
- **Task:** predict whether Sales are **below 7** (thousand units) using CompPrice, Income, and Advertising.

Variables

- Sales: unit sales (in thousands) at each location
- CompPrice: price charged by a competitor (in dollars)
- Income: community income level (in thousands of dollars)
- Advertising: local advertising budget (in thousands of dollars)

	Sales	CompPrice	Income	Advertising	Population	Price	ShelveLoc	Age	Education
1	9.50	138	73	11	276	120	Bad	42	17
2	11.22	111	48	16	260	83	Good	65	10
3	10.06	113	35	10	269	80	Medium	59	12
4	7.40	117	100	4	466	97	Medium	55	14
5	4.15	141	64	3	340	128	Bad	38	13
6	10.81	124	113	13	501	72	Bad	78	16

Urban US

1	Yes	Yes
2	Yes	Yes
3	Yes	Yes
4	Yes	Yes
5	Yes	No
6	No	Yes

Carseats (binary classification)

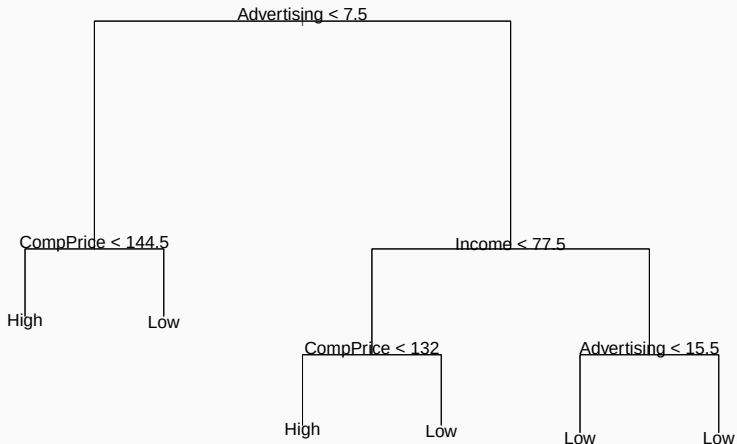


Figure 1: Classification tree for low vs high sales in Carseats

Recursive Binary Splitting

Algorithm

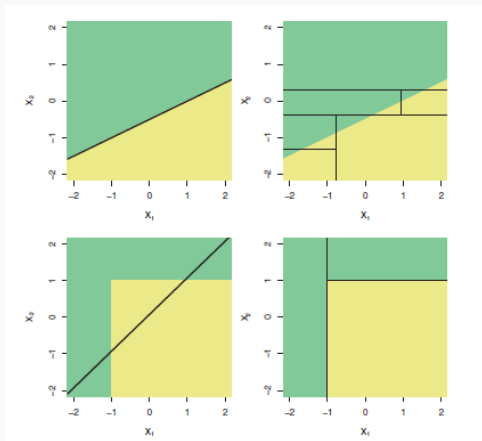
1. Split the feature space into **non-overlapping rectangular regions**.
2. Pick splits that **reduce node impurity** (make nodes more homogeneous).
3. Label each region using the **most common class** inside it.
4. Stop when more splitting **does not improve purity**.

Note: The method is **top-down and greedy**, so it can have **high variance**.

Classification with trees or regression?

- The choice depends on how classes relate to the features.
- **Linear models** work best when classes are separated by a **straight-line boundary**.
- **Trees** work best when the separation can be described by **rectangular (axis-aligned) regions**.

Figure: diagonal boundaries favor regression models, box-shaped regions favor trees.



Package tree

Used to **fit (grow) and visualize classification trees**.

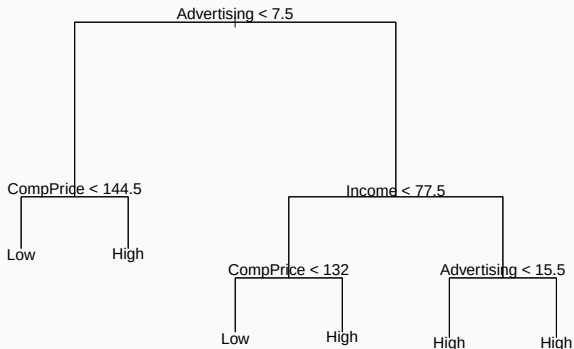
- `tree()` fits a classification tree model
- `split` specifies the **node impurity measure** ("deviance" or "gini")

```
fit_tree <- tree(  
  formula,  
  data,  
  split = c("deviance", "gini")  
)
```

```
plot(fit_tree)  # tree structure  
text(fit_tree) # add split rules and terminal node predictions to the tree plot
```

Carseats (binary classification)

```
tree.carseats <- tree(  
  factor(Sales < 7, labels = c("High", "Low")) ~ CompPrice + Income + Advertising,  
  data = ISLR::Carseats)  
  
plot(tree.carseats)  
text(tree.carseats, pretty = 0, cex = 1.4)
```



How can we improve tree models?

We can improve tree models by controlling the **bias–variance trade-off**.

Methods:

1. **Pruning**

Cut branches using cross-validation or regularization.

2. **Bagging**

Average predictions from bootstrapped trees.

3. **Random Forests**

Average predictions from bootstrapped trees while randomly selecting features at each split.

4. **Boosting**

Builds small trees sequentially; mainly reduces bias, but can be regularized to control variance.

Pruning

Cost-Complexity Pruning: Steps (tree package)

Goal: reduce variance by simplifying an overgrown tree.

0. Fit a classification tree model: `fit_tree`
1. **Cross-validate the tree** to find the optimal number of terminal nodes
 - based on deviance or misclassification error

```
# Option 1: deviance-based pruning
cv_tree <- cv.tree(fit_tree, method = "deviance")
# Option 2: misclassification-based pruning
cv_tree <- cv.tree(fit_tree, method = "misclass")
# cv.tree() uses prune.tree by default
```

2. **Prune the tree** to the optimal size
 - select the number of nodes with the smallest CV error (`dev`)

```
best_size <- cv_tree$size[which.min(cv_tree$dev)]
pruned_tree <- prune.tree(fit_tree, best = best_size)
```

3. **Predict on new data**
 - `predict.tree` returns class labels or probabilities

```
predict(pruned_tree, newdata, type = "class")
```

Example spam data

- The spam data are used to classify emails as **spam or not spam** using text-based features and the `type` is the response variable.

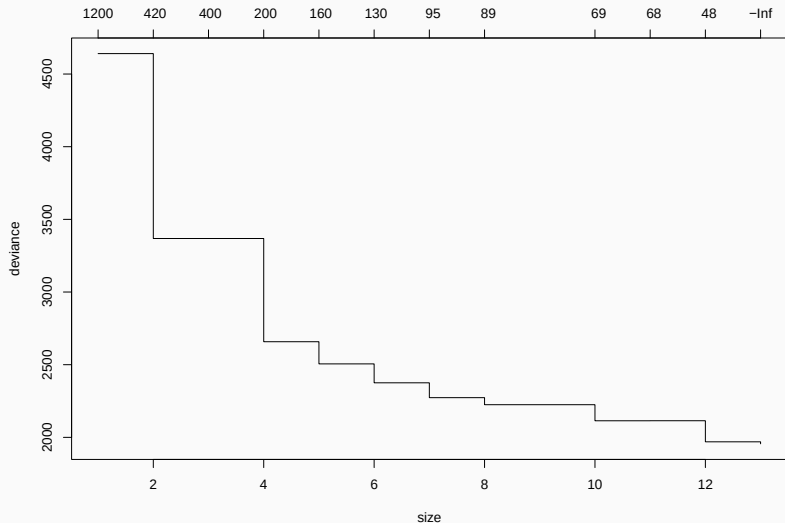
Cross-validate the tree

```
set.seed(12)

spam.tree <- tree(type ~ ., data = spam_train) # step 0
cv_tree <- cv.tree(spam.tree, method = "deviance") # step 1
```

CV error plot

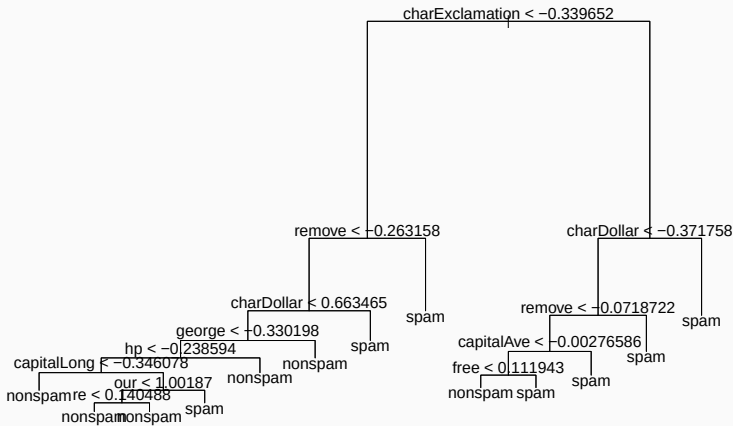
```
plot(cv_tree)
```



Pruned tree with `prune.tree()`

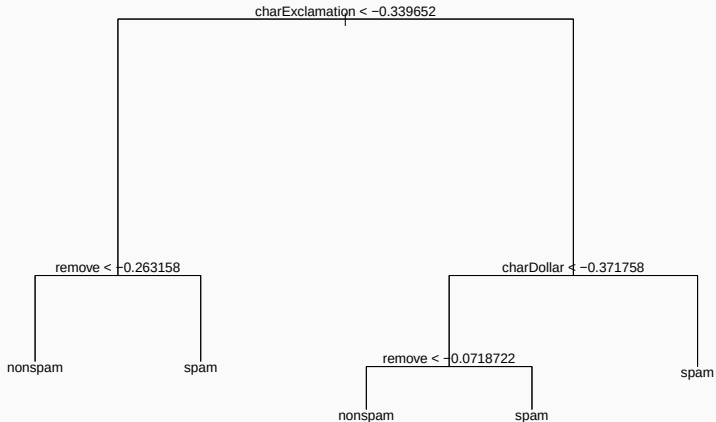
step 2

```
best_size <- cv_tree$size[which.min(cv_tree$dev)]  
spam.prune <- prune.tree(spam.tree, best = best_size)  
plot(spam.prune); text(spam.prune, cex = 1.2)
```



Manually prune the tree to a specific size without cross-validation `prune.tree()`

```
pruned_tree <- prune.tree(spam.tree, best = 5)
# 5. Plot the pruned tree
plot(pruned_tree)
text(pruned_tree)
```



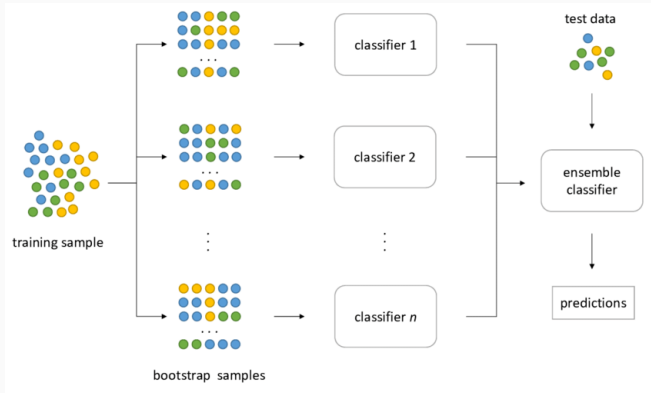
Bagging and Random Forests

Algorithm

- **Bagging**

1. Fit classification trees to "B bootstrap samples" of the training data.
2. Combine predictions across all trees by "majority vote".
3. Use "Out-of-Bag (OOB) observations" to estimate the validation error.

- **Random Forests:** Same as bagging, but with one extra step: At each split, randomly select a subset of predictors as candidates.



Bagging vs Random Forests

Bagging

- Considers **all predictors** at each split
- Strong predictors dominate many trees
- Trees are **highly correlated**
- Variance reduction is limited

Random Forests

- Considers a **random subset of predictors** at each split
- Gives all predictors a chance to contribute.
- Reduces the dominance of very strong predictors
- Trees are **less correlated (decorrelated)**
- Usually leads to **lower variance** and better generalization

Out-of-Bag (OOB) Error Rate

On average, about **1/3 of observations** are left out of each bootstrap sample.

- These **out-of-bag (OOB)** observations are used to estimate validation error.
- Provides an **internal estimate of test error**.
- Often **no separate cross-validation** is needed.
- **Computationally efficient**.

Package randomForest for bagging and random forest

```
fit <- randomForest(formula, data, ntree = 500,  
                    mtry = <number of predictors considered at each split>,  
                    importance = TRUE)  
varImpPlot(fit)  
predict(fit, newdata, type = "prob") # classification only
```

- mtry: number of predictors randomly considered at each split (random forest default: $\sqrt{p}$ and bagging: p (all predictors))
- ntree is tuning parameter (number of trees). larger values usually increase stability and generally do not cause overfitting, but increase computation time
- importance = TRUE enables full variable importance
 - Permutation importance: Mean Decrease Accuracy
 - Impurity importance: Mean Decrease Gini
 - varImpPlot(fit) displays the available importance measures.
- predict(..., type = "prob"): returns class probabilities
 - default type = "response" returns class labels

Random Forest on spam Data

```
set.seed(1)

spam.rf <- randomForest(
  type ~ .,
  data = spam_train,
  ntree = 500,
  importance = TRUE
)
```

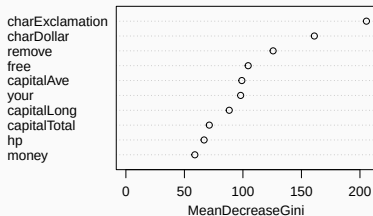
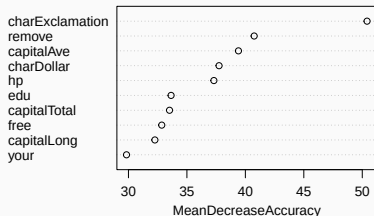
Fit a random forest with 500 trees, using a random subset of predictors at each split.

Variable Importance

Unlike a single decision tree, a random forest does not have one simple tree structure to interpret.

- For classification, predictions are made by combining votes across many trees.
- Interpretation is done by assessing which predictors are most useful across the forest.
- This is summarized using **variable importance measures**.
- Importance can be visualized with variable importance plots, `varImpPlot()`.

```
varImpPlot(spam.rf, n.var = 10, main = "")
```



How to Interpret the Plot

The plot shows the **10 most important predictors** in the random forest.

- **MeanDecreaseAccuracy (left):**

How much prediction accuracy decreases when a variable is randomly shuffled.

Larger decrease = more important.

- **MeanDecreaseGini (right):**

How much a variable reduces node impurity across trees.

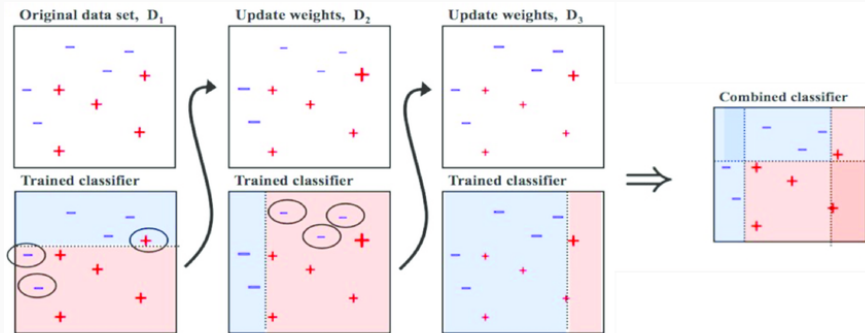
Larger value = more important.

- Importance summarizes **overall predictive influence**, not a single rule.

Boosting

1. Fit a **weak learner** to the training data.
2. Add learners sequentially, where each new learner focuses on previous errors.
 - **AdaBoost:** gives more weight to misclassified observations.
 - **Gradient boosting:** each new learner tries to correct the current model's errors.
 - **XGBoost:** an efficient, regularized version of gradient boosting.
3. Combine all weak learners into one **ensemble model**.

AdaBoost



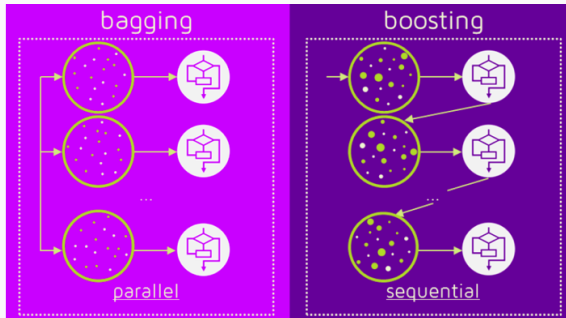
Boosting vs bagging/random forest

Bagging / Random Forests

- Trees are trained **independently**
- Algorithm is **parallel by design**
- No dependence between trees

Boosting

- Trees are trained **sequentially**
- Each tree depends on previous errors



XGBoost in R

```
library(xgboost)

spam_train <- as.data.frame(spam_train)
spam_train$type <- factor(spam_train$type)

x_train <- model.matrix(type ~ ., data = spam_train)[, -1]
y_train <- ifelse(spam_train$type == levels(spam_train$type)[2], 1, 0)

dtrain <- xgb.DMatrix(data = x_train, label = y_train)

params <- list( objective = "binary:logistic",
  eval_metric = "error", max_depth = 3,
  eta = 0.1, subsample = 0.8,
  colsample_bytree = 0.8)

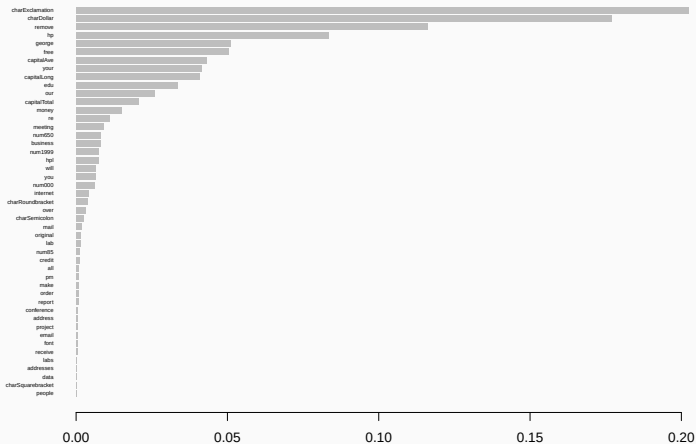
set.seed(1)

xgb_fit <- xgb.train( params = params,
  data = dtrain, nrounds = 100, verbose = 0)

xgb_cv <- xgb.cv( params = params, data = dtrain,
  nrounds = 100, nfold = 5, verbose = 0)
```

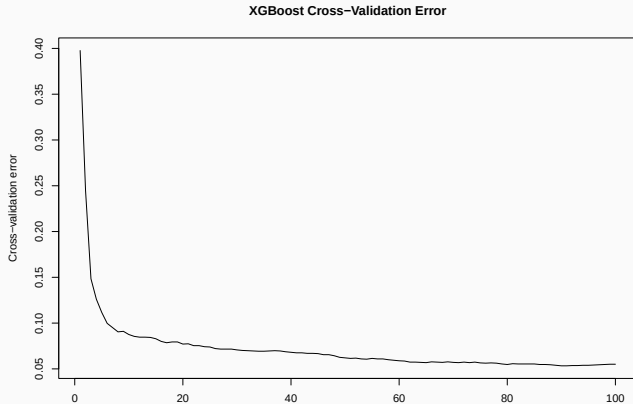
Plot train object: Variable importance

```
importance <- xgb.importance(  
  feature_names = colnames(x_train), model = xgb_fit )  
xgb.plot.importance(importance)
```



XGBoost CV error plot

```
# Plot cross-validation error  
plot(xgb_cv$evaluation_log$iter,  
     xgb_cv$evaluation_log$test_error_mean,  
     type = "l",  
     xlab = "Number of trees",  
     ylab = "Cross-validation error",  
     main = "XGBoost Cross-Validation Error")
```



Support Vector Machines (SVM)

Support Vector Methods for Binary Classification

Support vector methods classify observations by finding a boundary with a large margin between classes.

1. Maximal margin classifier

- Uses a linear boundary.
- Requires the classes to be perfectly separable.

2. Support vector classifier

- Uses a linear boundary.
- Allows some observations inside the margin or misclassified.

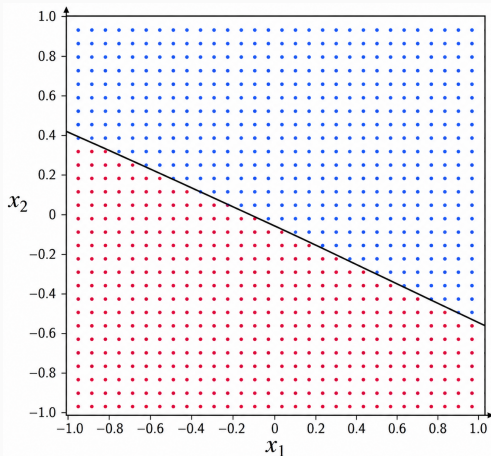
3. Support vector machine

- Uses kernels to create non-linear decision boundaries.

Hyperplane

A **hyperplane** is the decision boundary that **divides the feature space into two parts**.

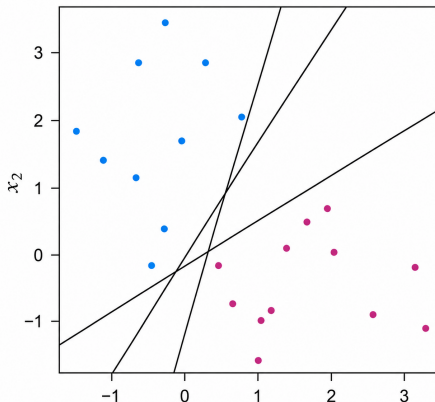
- In **two dimensions**, a hyperplane is simply a **line**.



Separating Hyperplane

A **separating hyperplane** perfectly separates the two classes.

- Many separating hyperplanes may exist.
- The choice is **not unique**.
- Small changes in the data can change the hyperplane
 - leads to **high variance**.



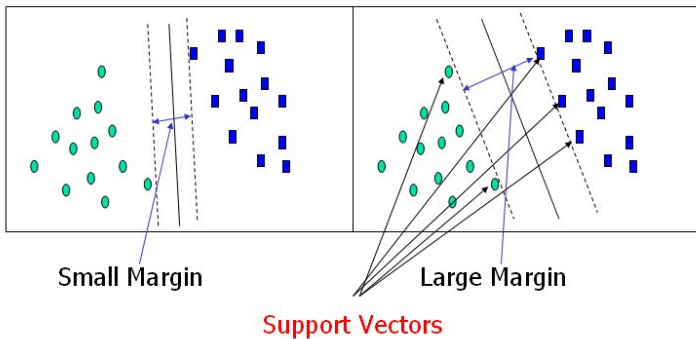
So what?

- Perfect separation alone is not enough; we need a rule to choose a **stable** hyperplane.
- This sets up **why the maximum-margin idea is needed** in the next slide.

Maximal Margin Classifier

Identifies the separating hyperplane that **maximizes the margin** between classes (farthest from the closest points).

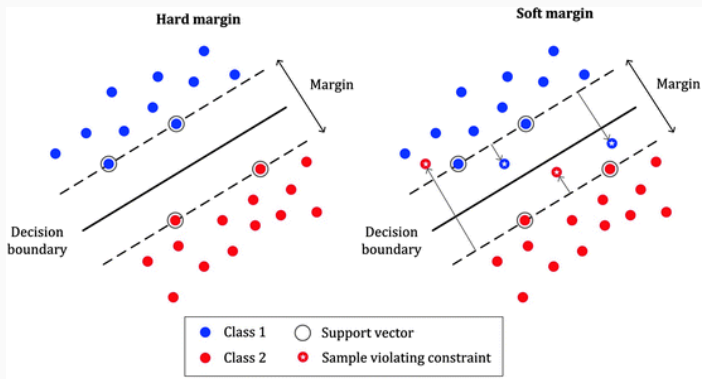
- The points lying on the margin are the **support vectors**.
- Works **only when the classes are perfectly separable**.



Support Vector Classifier (SVC)

Allows for violations of the margin (soft margin)

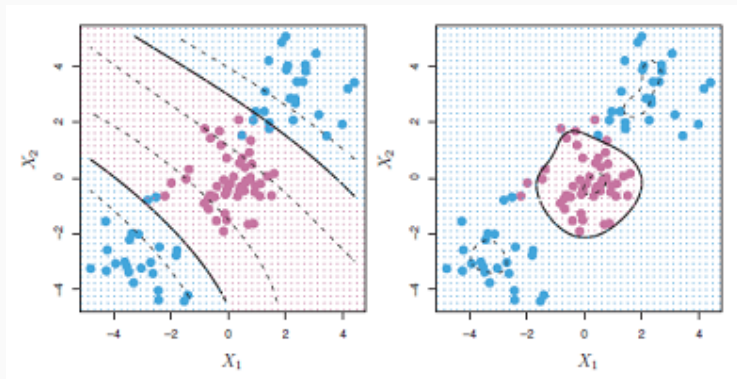
- budget for violations is called (controlled by) the *cost* (C)
 - C controls how strict the margin is.
 - Large C : fewer violations, narrower margin.
 - Small C : more violations allowed, wider margin.
- cases on the wrong side of the hyperplane contribute to the cost



Support Vector Machines (SVM)

Kernels allow SVMs to separate data using **nonlinear boundaries**.

- **Polynomial kernel (left):** fits smooth, curved decision boundaries.
- **Radial (RBF) kernel (right):** fits very flexible, localized boundaries.



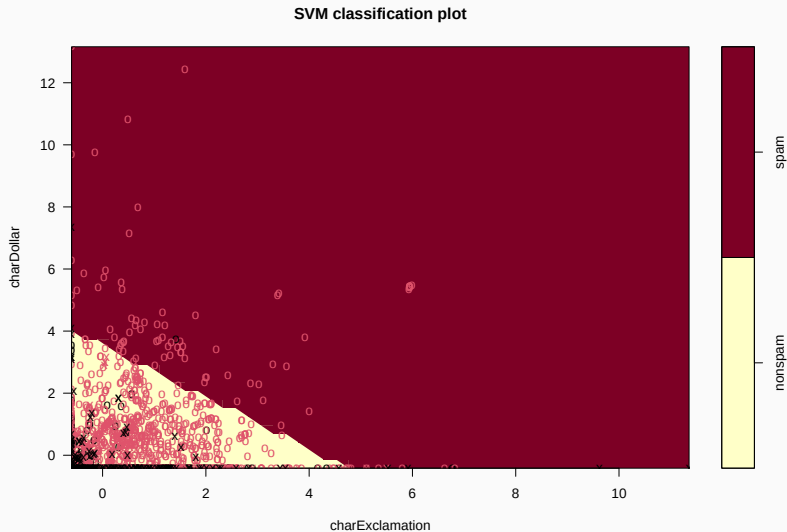
SVM with linear kernel

```
# Tune a Support Vector Machine with a linear kernel
spam.svm.1 <- tune.svm(
  type ~ .,          # formula: predict email type using all predictors
  data  = spam_train, # spam dataset
  kernel = "linear", # use a linear kernel (straight hyperplane)
  cost   = 1:5        # try different values of the cost parameter
)
```

Parameter tuning of 'svm':

- sampling method: 10-fold cross validation
- best parameters:
 - cost
 - 3
- best performance: 0.05913043

Classification Plot (Linear SVM)



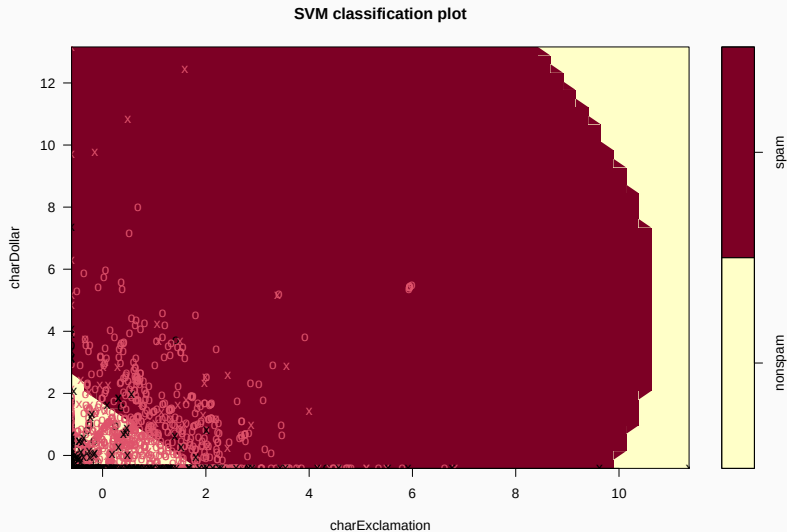
SVM with Radial Kernel

```
# Tune a Support Vector Machine with a radial (RBF) kernel
spam.svm.r <- tune.svm(
  type ~ .,          # predict email type using all predictors
  data  = spam_train, # spam dataset
  kernel = "radial", # radial (RBF) kernel for nonlinear boundaries
  cost   = 1:5        # try different values of the cost parameter
)
```

Parameter tuning of 'svm':

- sampling method: 10-fold cross validation
- best parameters:
cost
3
- best performance: 0.04898551

Classification Plot (Radial SVM)



Predictions test set

We compare **test-set misclassification error rates** across different models.

	prune	randomforest	boosting	svm.linear	svm.radial
error rate	0.0999	0.0504	0.053	0.0599	0.0582

Interpretation

- The pruned tree has the highest error rate, **0.0999**.
 - It has the weakest predictive performance.
- Random forest and boosting have the lowest error rates, both **0.0495**.
 - They give the best test-set performance among these models.
- Random forest and boosting perform better than a single pruned tree.
 - Combining many trees reduces error compared with using one tree.
- The radial SVM performs better than the linear SVM.
 - This suggests that a nonlinear boundary may work better than a simple linear boundary for this dataset.
- However, radial SVM is not the best model here.
 - It has an error rate of **0.0547**, which is slightly higher than random forest and boosting.

Conclusion: Random forest and boosting achieve the lowest test-set error rate, both at **4.95%**, making them the best-performing models in this comparison.

Pros and Cons of Classifiers

Logistic Regression - Robust to outliers in predictors - Can be unstable with highly correlated predictors - **Easy to interpret** (coefficients, odds ratios)

LDA / QDA - Generally **stable** classifiers - Sensitive to violations of **normality** and covariance assumptions - Less flexible than tree-based methods

Tree-Based Methods - Flexible and capture nonlinear relationships - Bias–variance trade-off controlled via **pruning, random forests, and boosting** - Interpretation is **moderate** (trees interpretable, ensembles less so)

Support Vector Machines (SVM) - Effective in high dimensions - Allow **nonlinear decision boundaries** via kernels - **Difficult to interpret** compared to parametric models